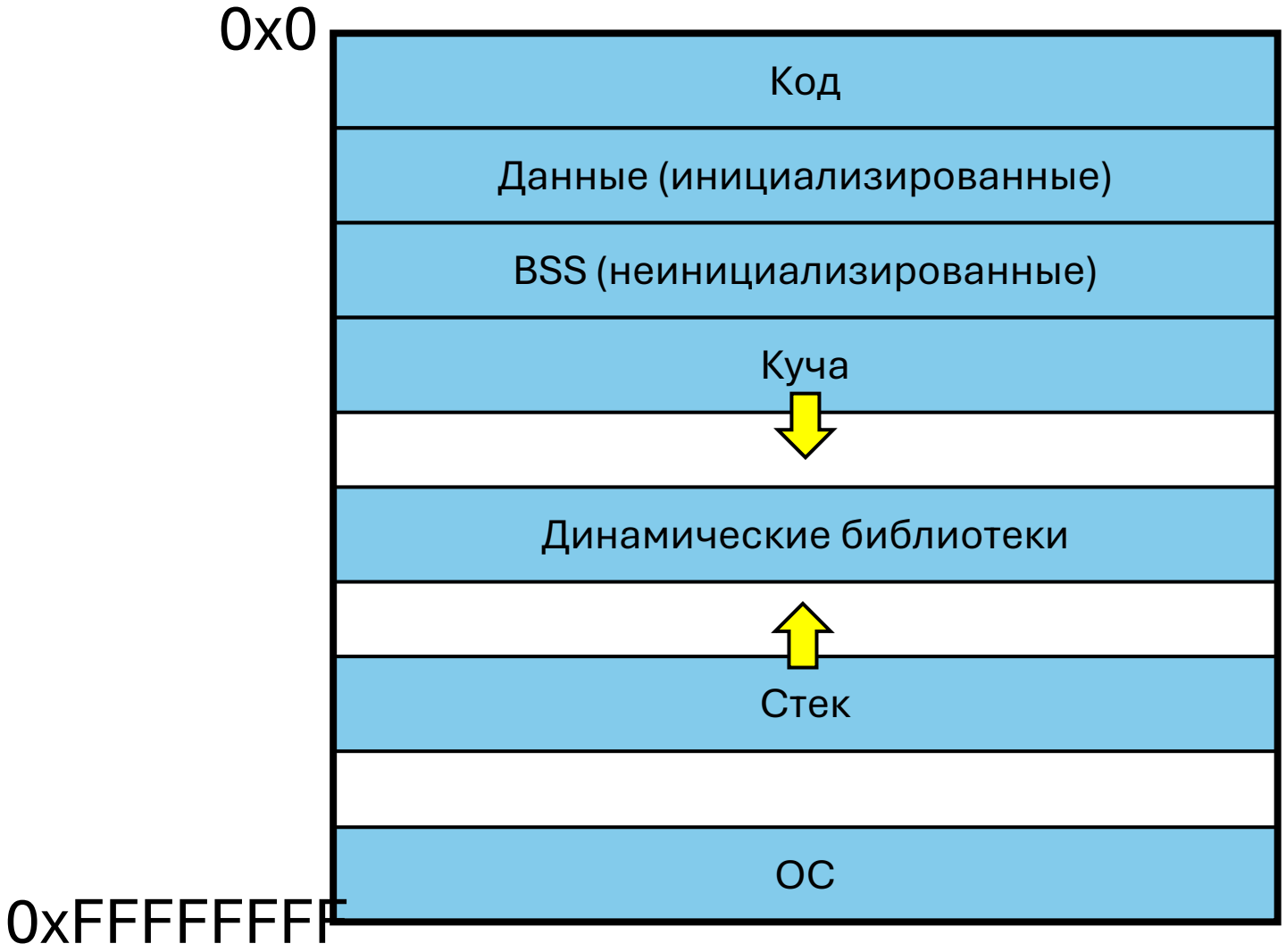
Как выиграть в игру?

```
sab@LAPTOP-B03PIUAN:.../Secret$ ./secret
Enter secret number:
123
You are so wrong!
```

Task 098

Секции программы



Напоминание: ассемблер (32-bit x86)

Регистр	Назначение
%eax	хранение результатов промежуточных вычислений
%ebx	хранения адреса (указателя) на некоторый объект в памяти
%ecx	счетчик
%edx	хранения результатов промежуточных вычислений
%esp	содержит адрес вершины стека
%ebp	указатель базы кадра стека
%esi	индекс источника
%edi	индекс приёмника

Команда	Назначение
mov *источник, назначение*	копирование *источника* в *назначение*
lea *источник, назначение*	помещает адрес *источника* в *назначение*
add *источник*, приёмник	*приёмник* = *приёмник* + *источник*
sub *источник*, приёмник	*приёмник* = *приёмник* - *источник*
push *источник*	поместить в стек
pop *назначение*	извлечь из стека
cmp *операнд_2, операнд_1*	*операнд_1 – операнд_2* и устанавливает флаги
jle *метка*	Переход если <=

Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

```asm
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

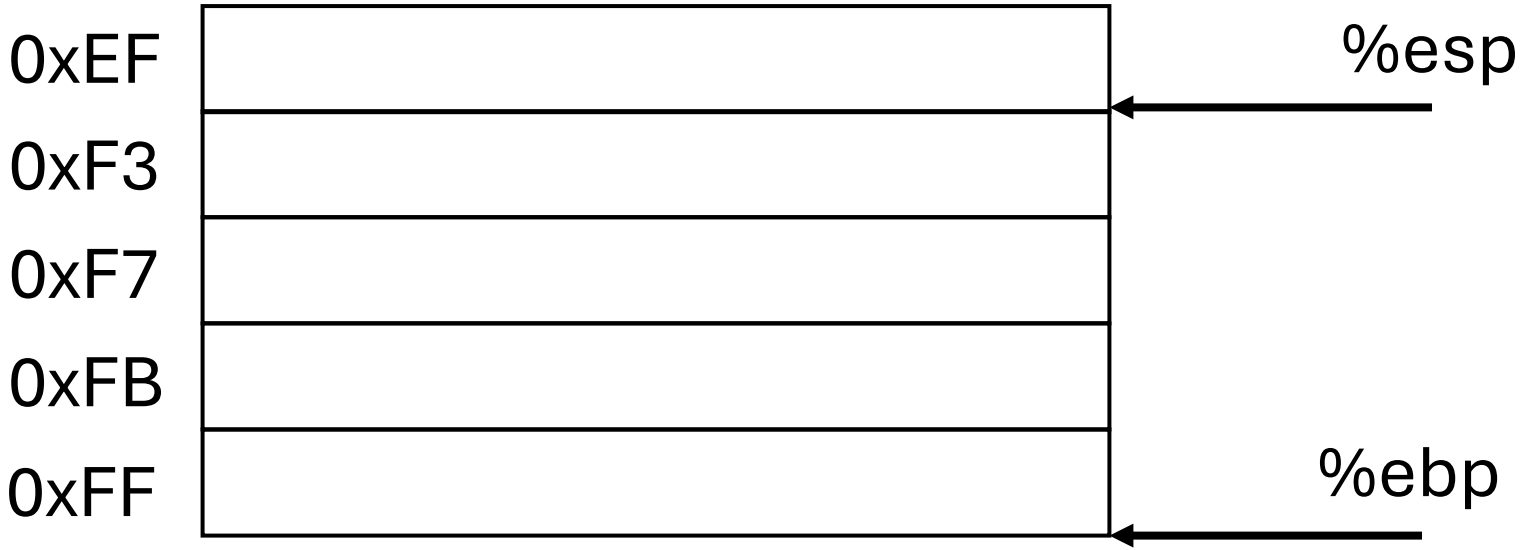
Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

```asm
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

Task 096

Стек - напоминание



Функции ассемблера

`leave`	prepares the stack for leaving a function. Equivalent to:
	`mov %ebp, %esp`
	`pop %ebp`

`call addr <fname>`	switches active frame to callee function. Equivalent to:
	`push %eip`
	`mov addr, %eip`

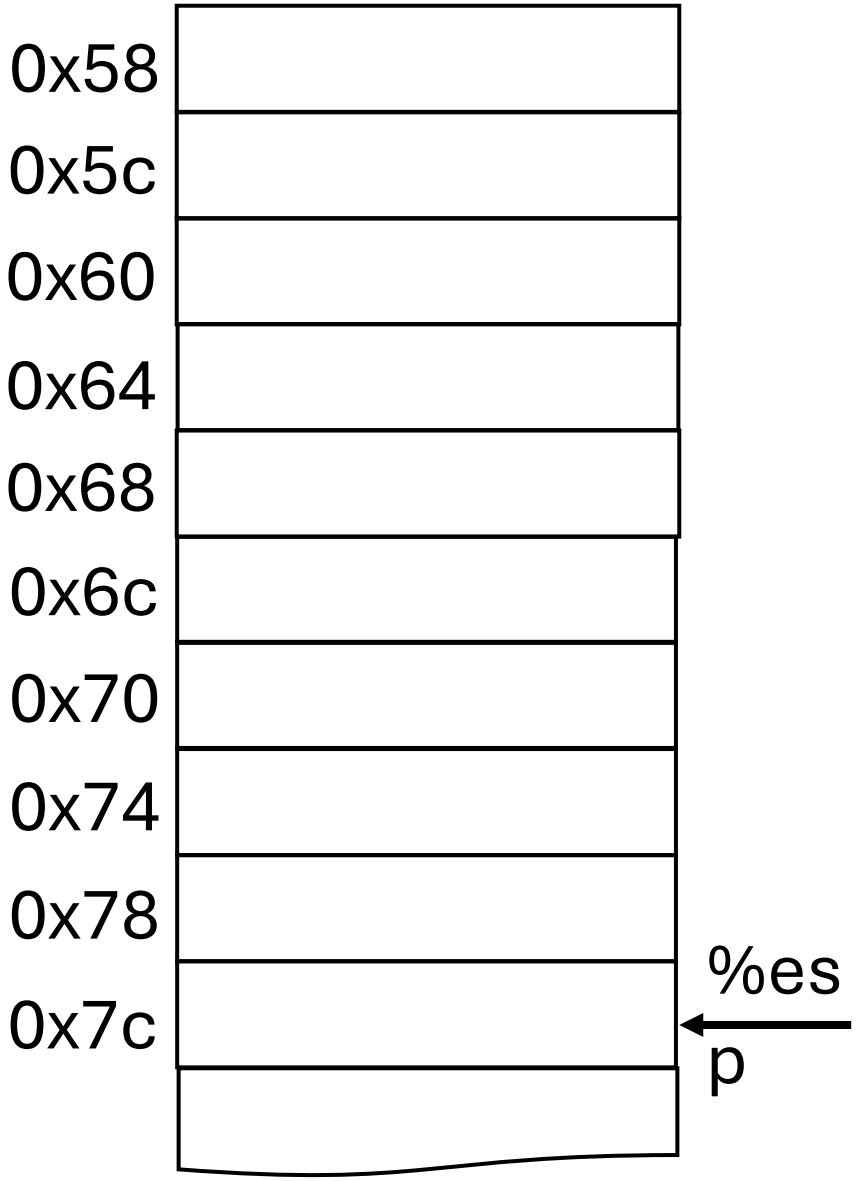
`ret`	restores active frame to caller function. Equivalent to: `pop %eip`

Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

%es	0xffffd07
%eb	0x0
%eip	0x565561c5

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

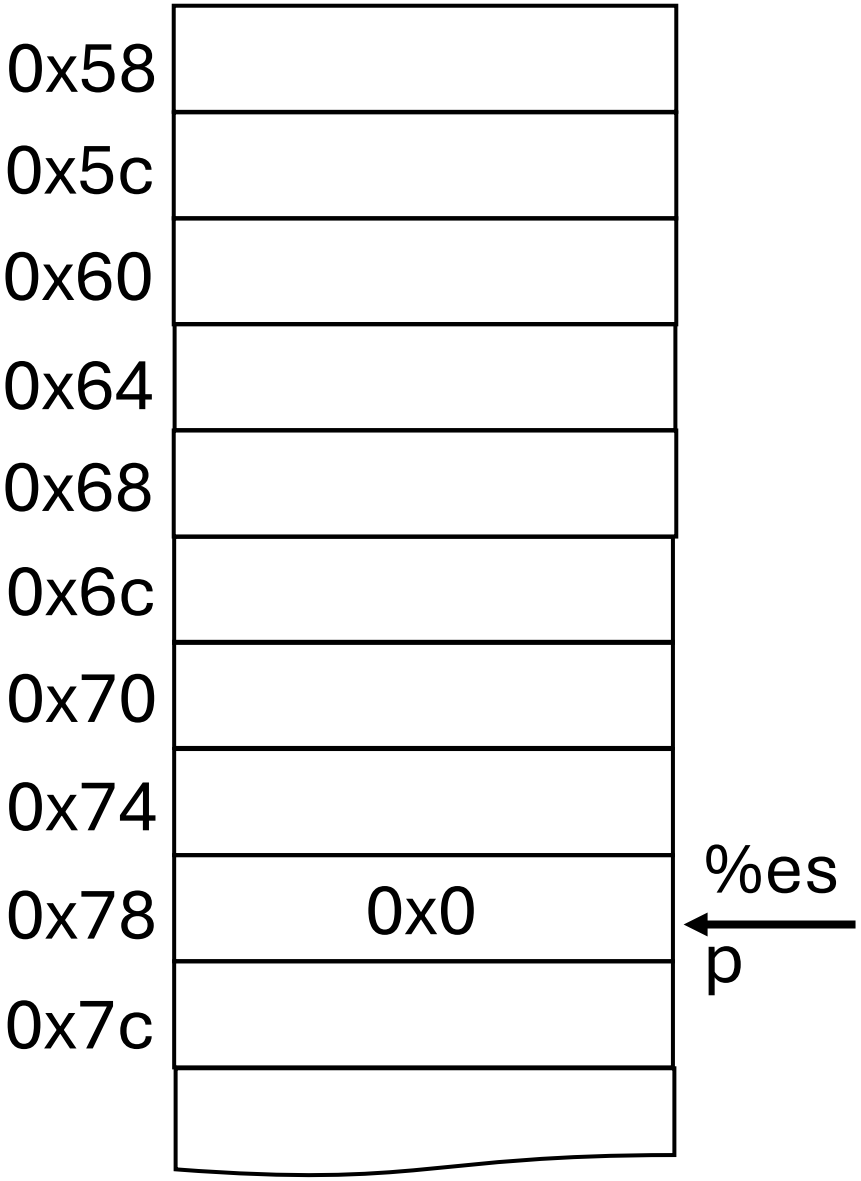


Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

%es	0xffffd078
%ebp	0x0
%eip	0x565561c9

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

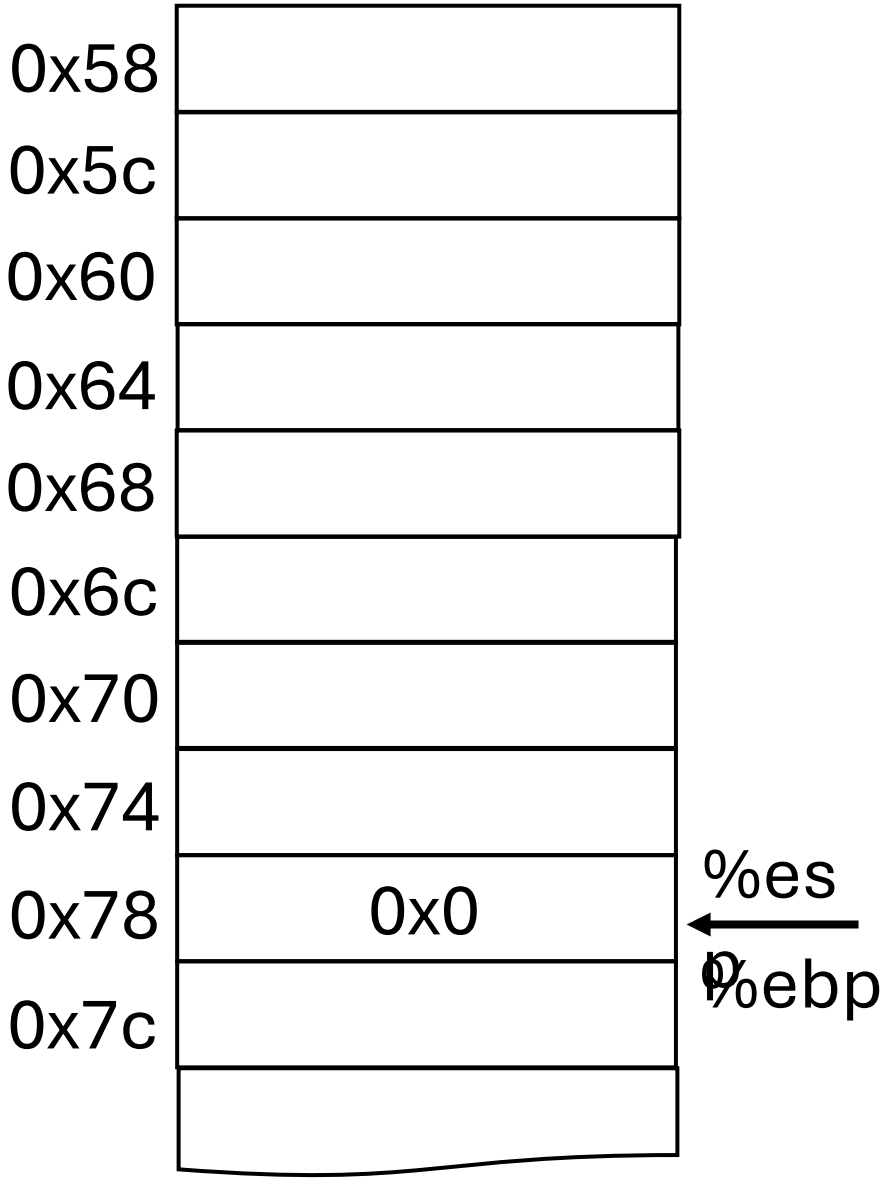


Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

%es	0xffffd078
%ebp	0xffffd078
%eip	0x565561ca

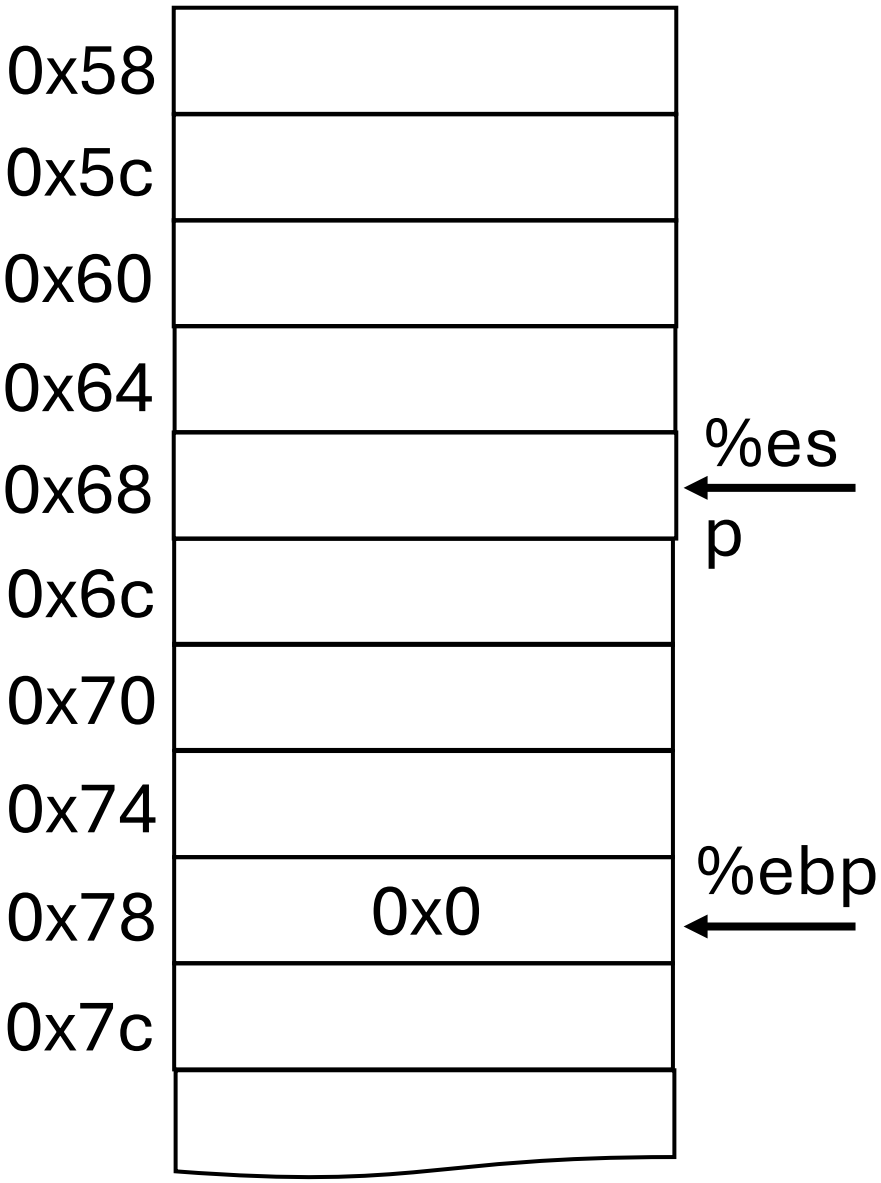


Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

%es	0xffffd068
%ebp	0xffffd078
%eip	0x565561cc

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

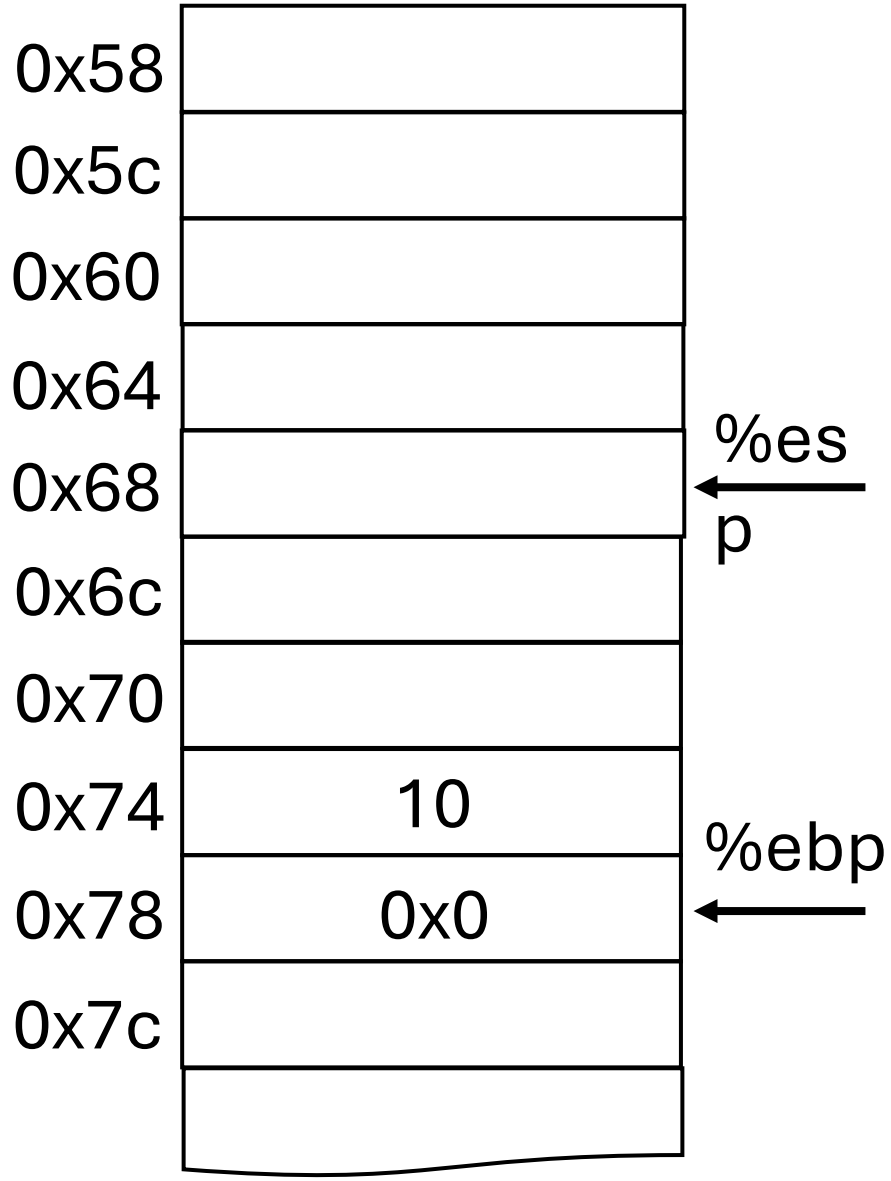


Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

%es	0xffffd06
%ebp	0xffffd078
%eip	0x565561d9

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

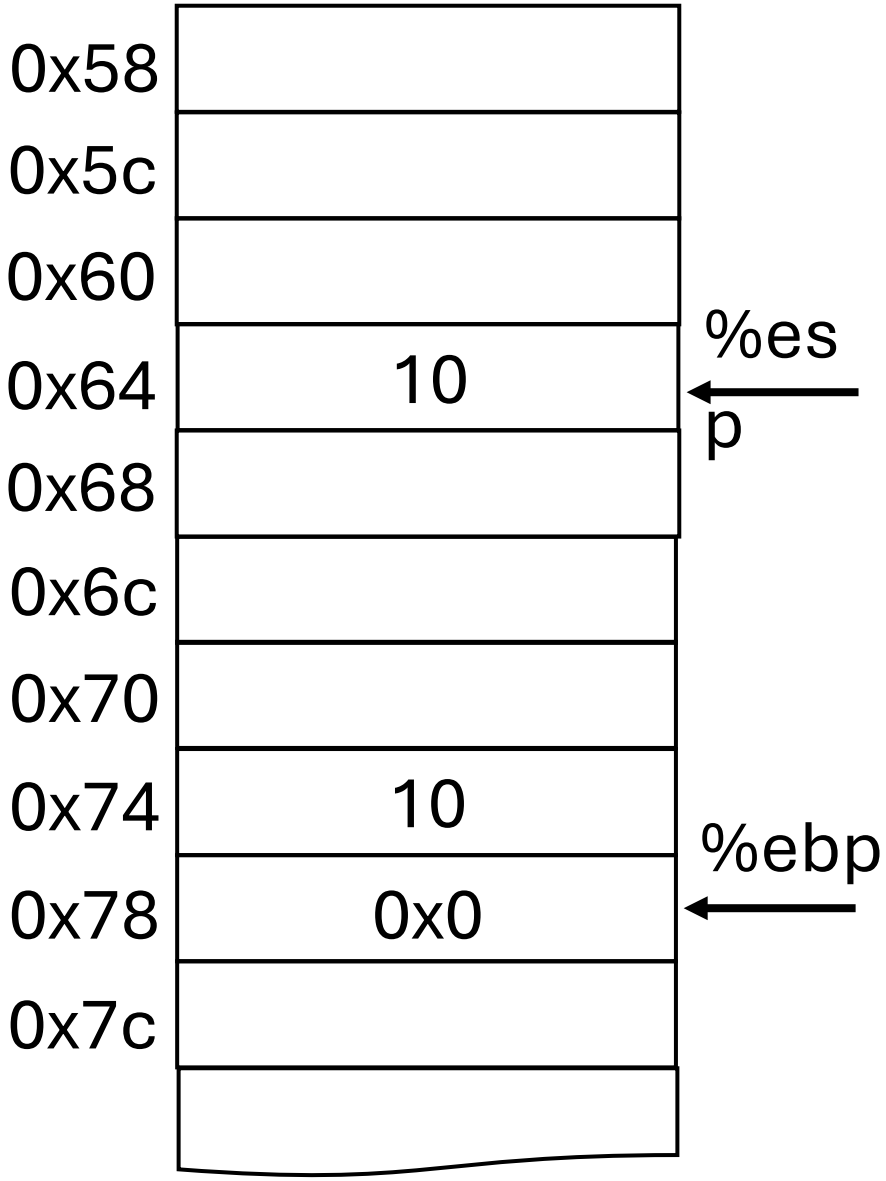


Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

%es	0xffffd06
%eb	0xffffd078
%eip	0x565561df



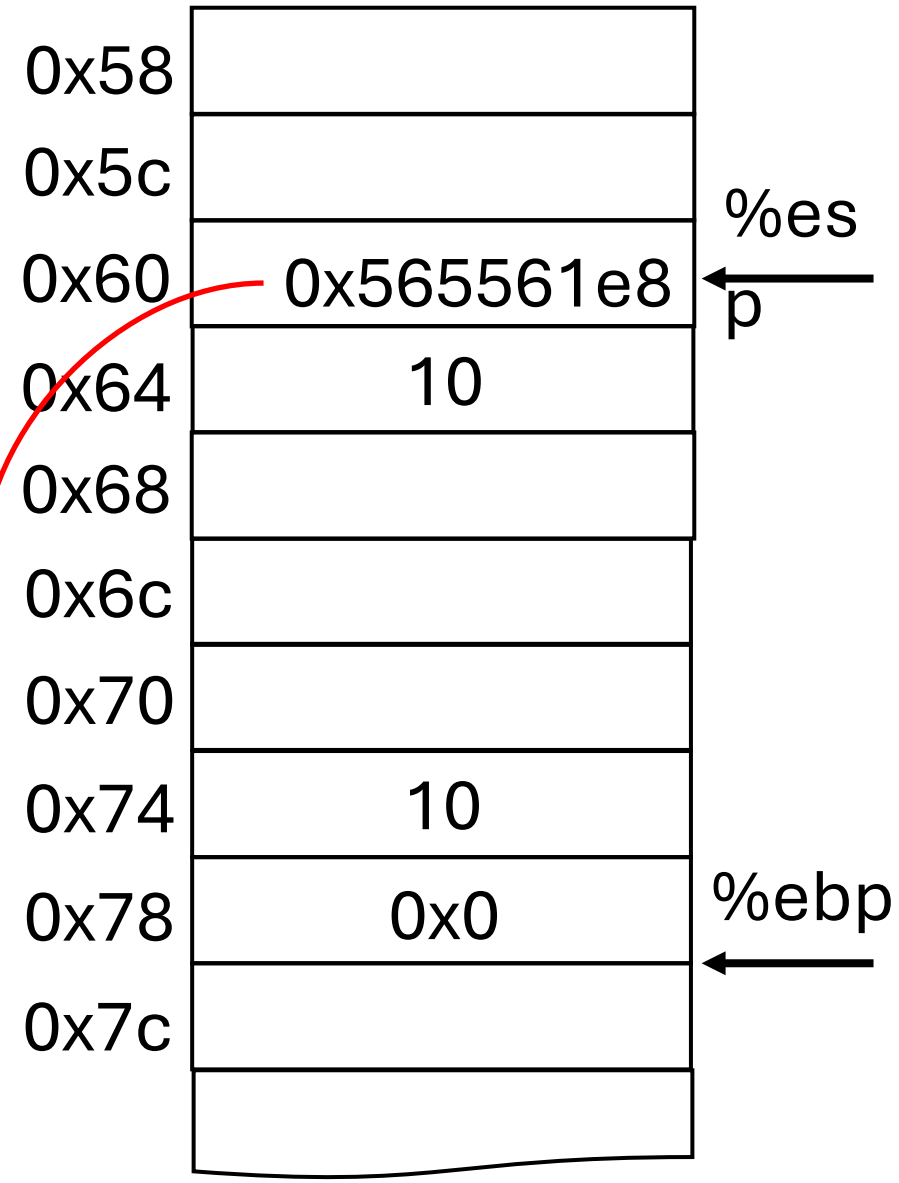
Вызов функции

```
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

Адрес

%es	0xffffd06
%ebp	0xffffd078
%eip	**0x565561b1**

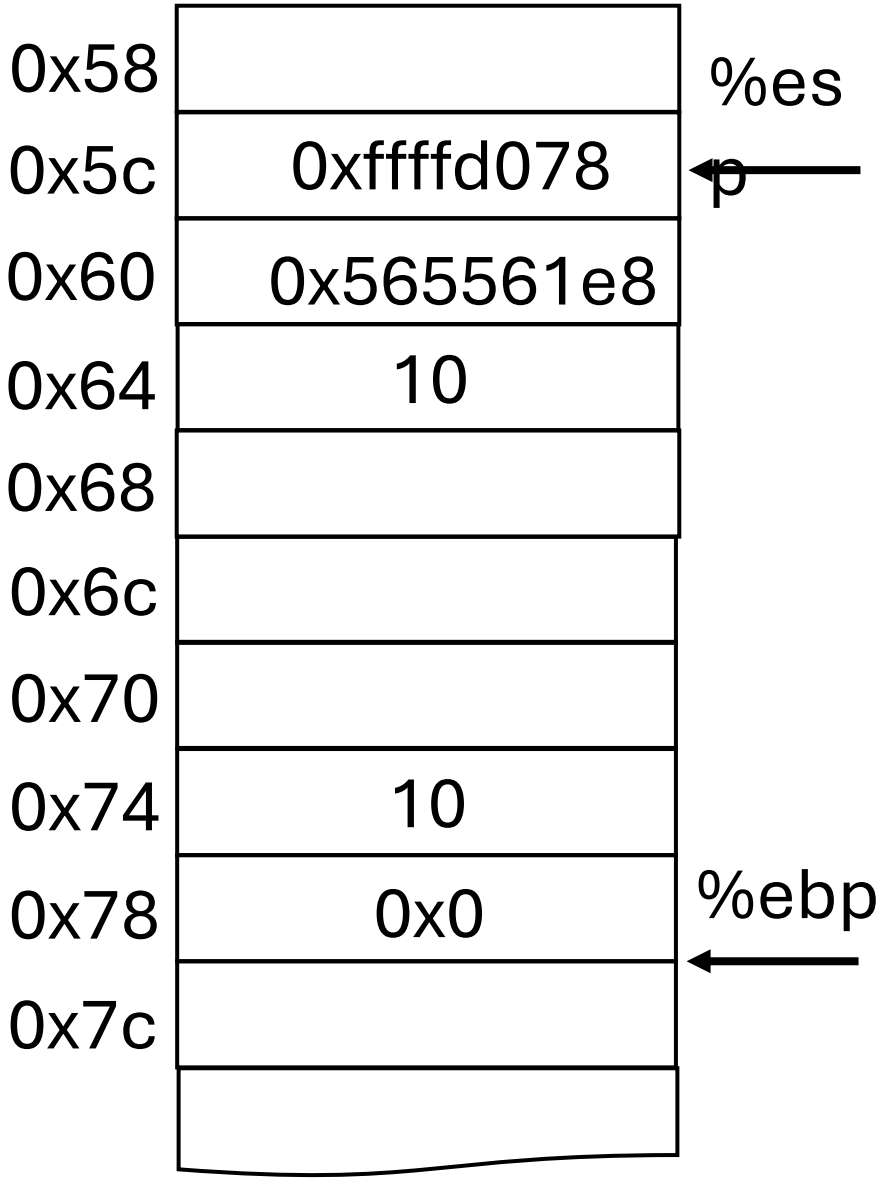


Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

%es	0xffffd05
%eb	0xffffd078
%eip	0x565561b1

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

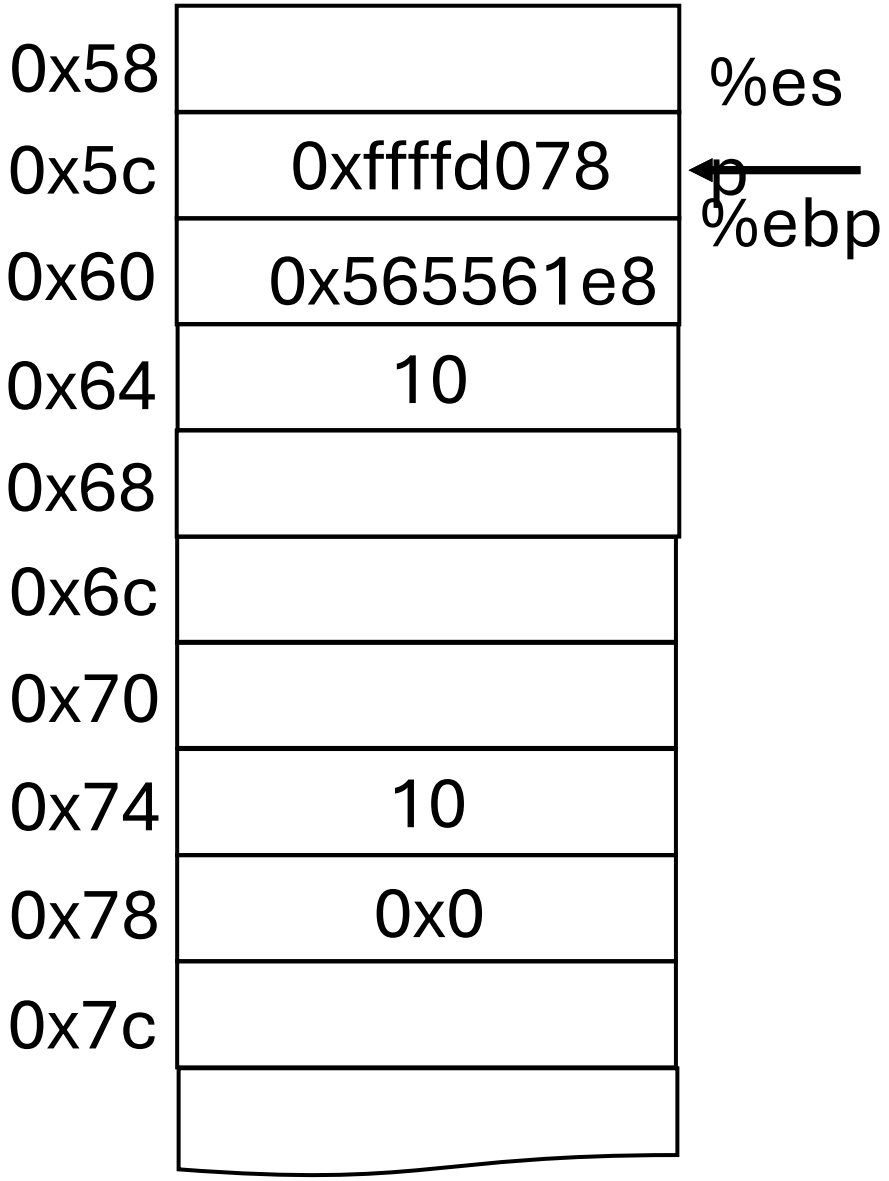


Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

%es	0xffffd05
%ebp	0xffffd05c
%eip	0x565561b2

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

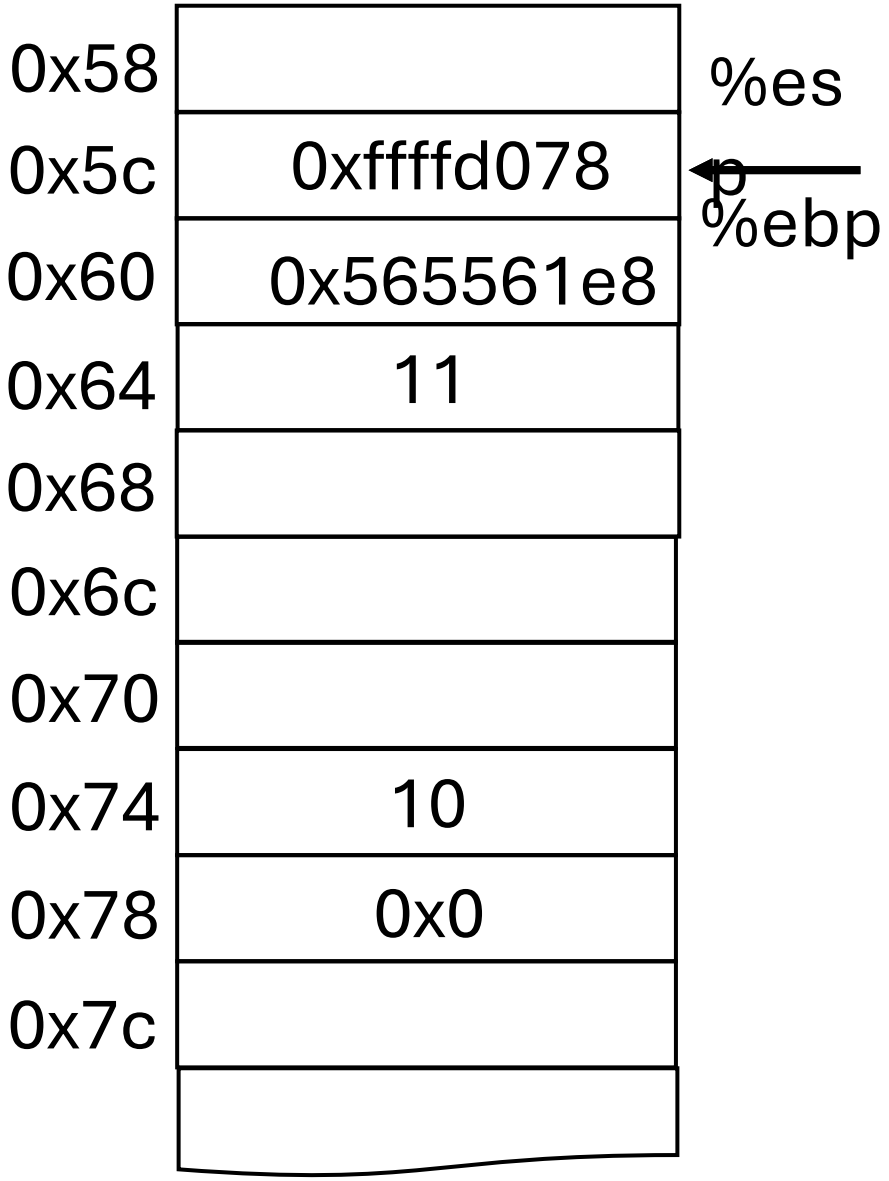


Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

%es	0xffffd05
%eb	0xffffd05c
%eip	0x565561b2

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

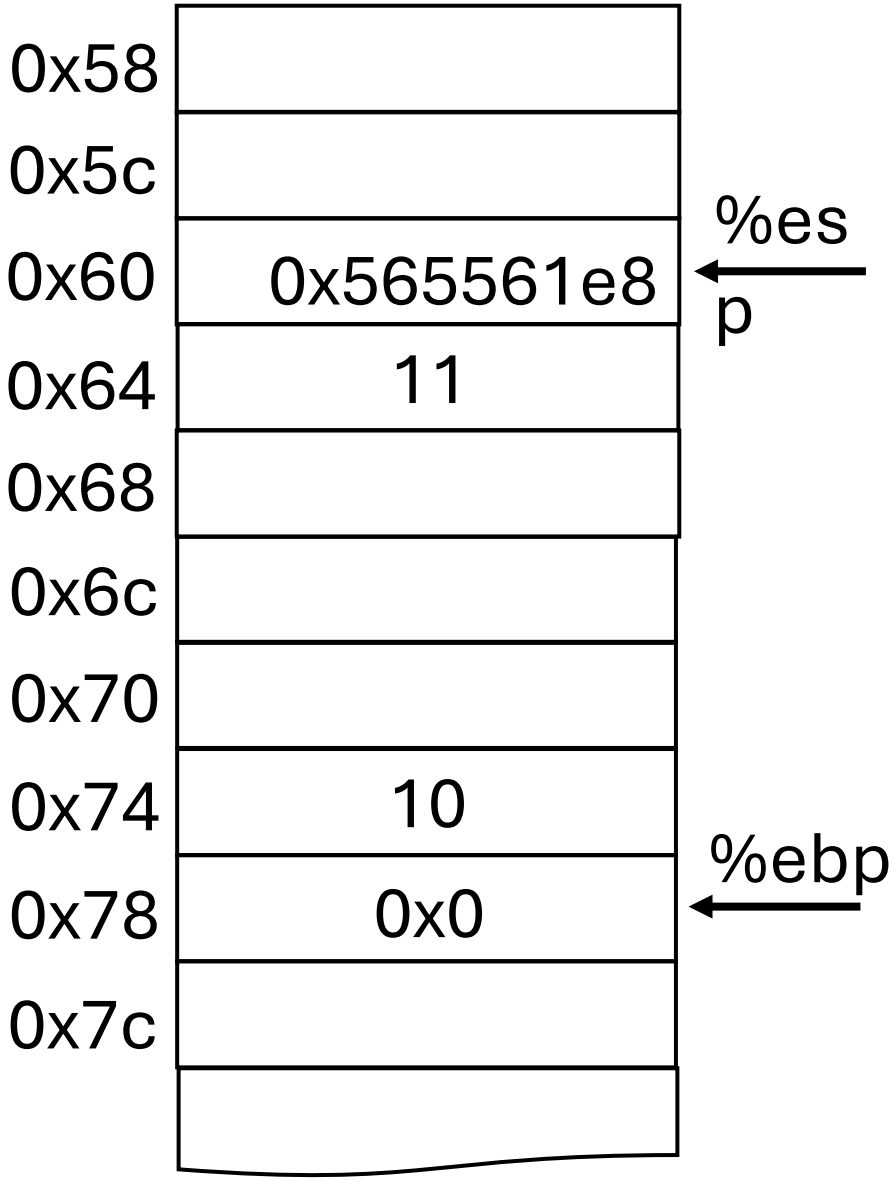


Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

%es	0xffffd06
%ebp	0xffffd078
%eip	0x565561b2

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

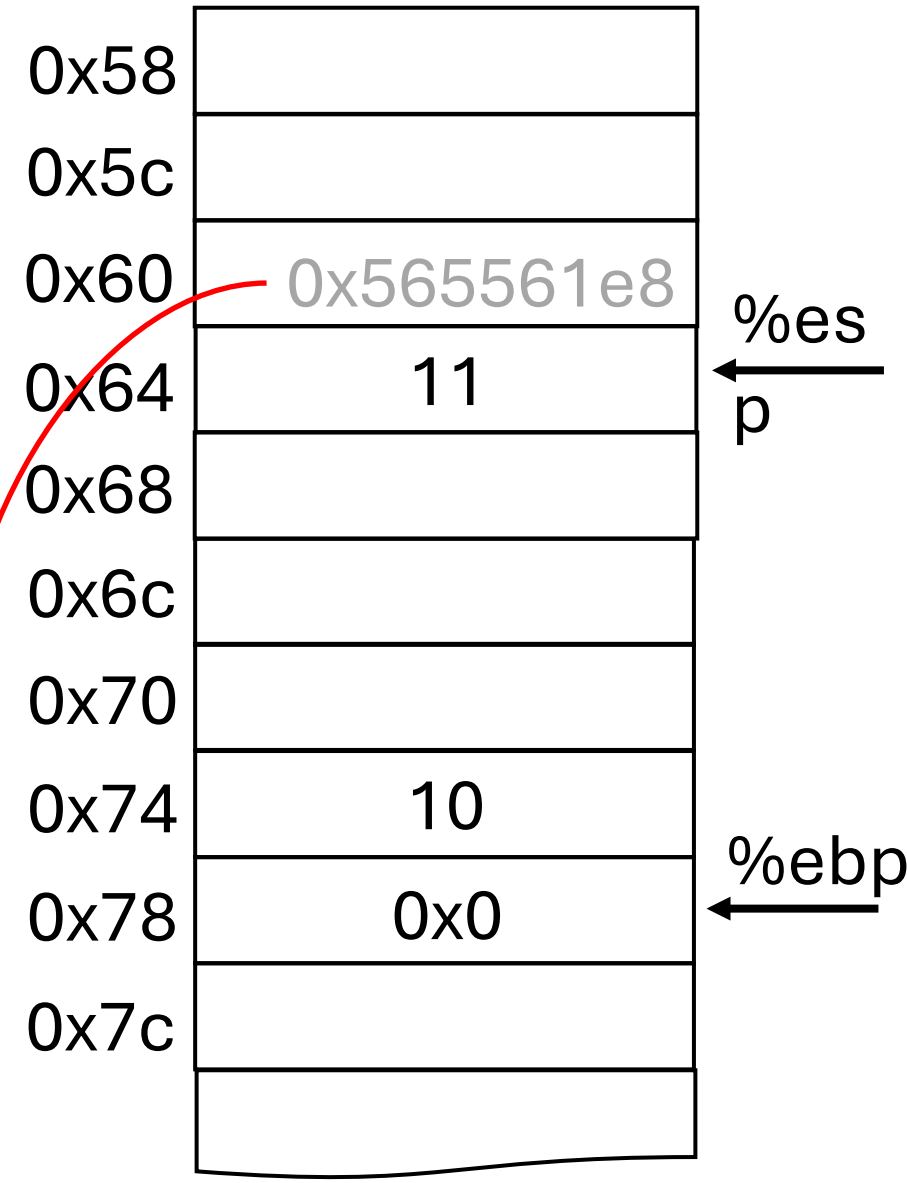


Вызов функции

```
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

%es	0xffffd06
%eb	0xffffd078
%eip	0x565561e8

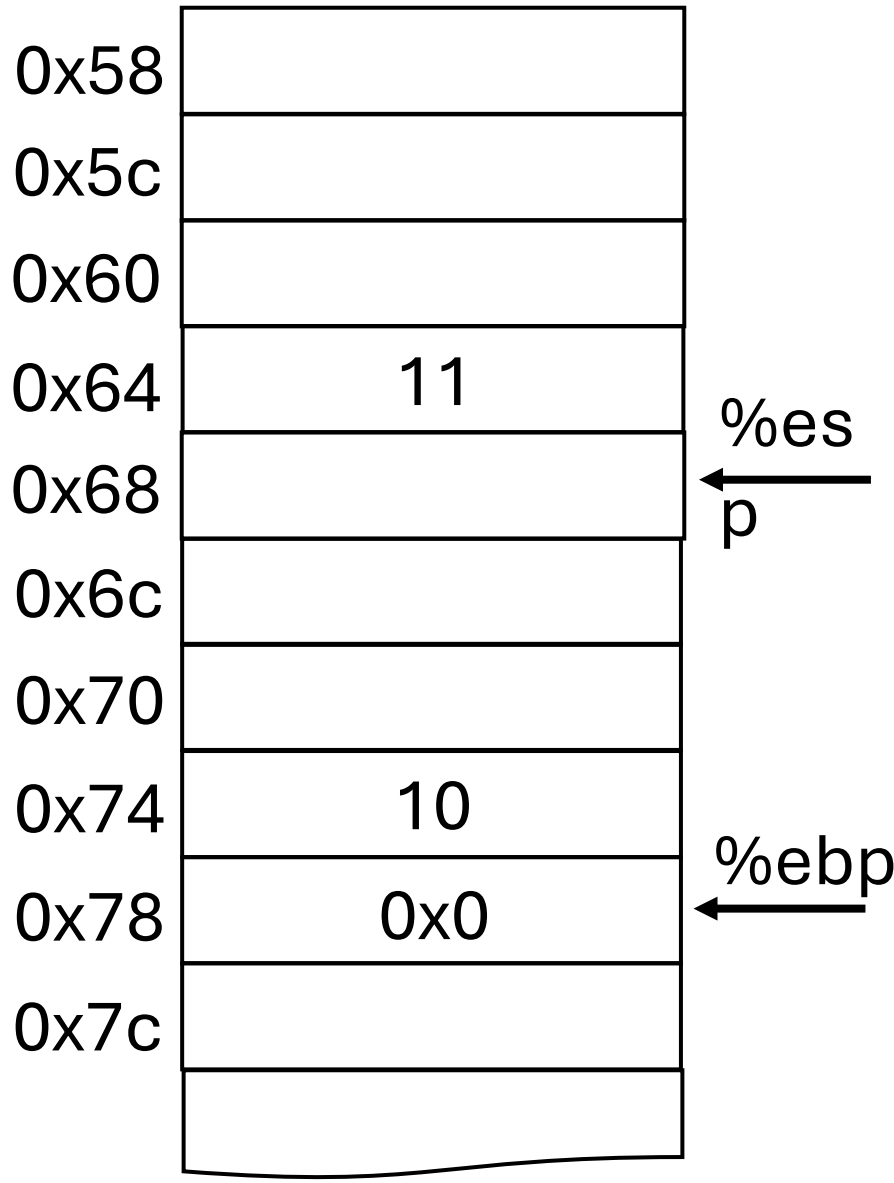


Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

%es	0xffffd068
%ebp	0xffffd078
%eip	0x565561e8

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

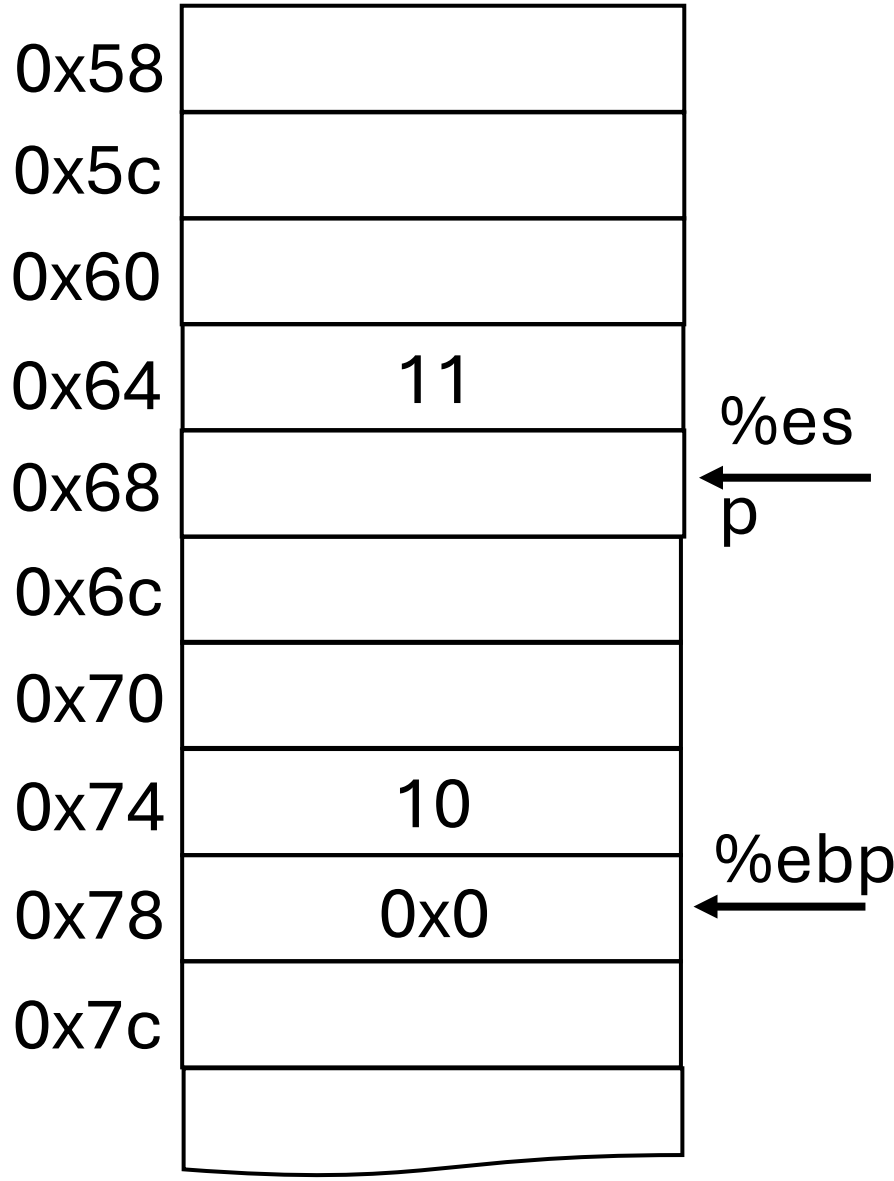


Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

%es	0xffffd068
%ebp	0xffffd078
%eip	0x565561eb

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

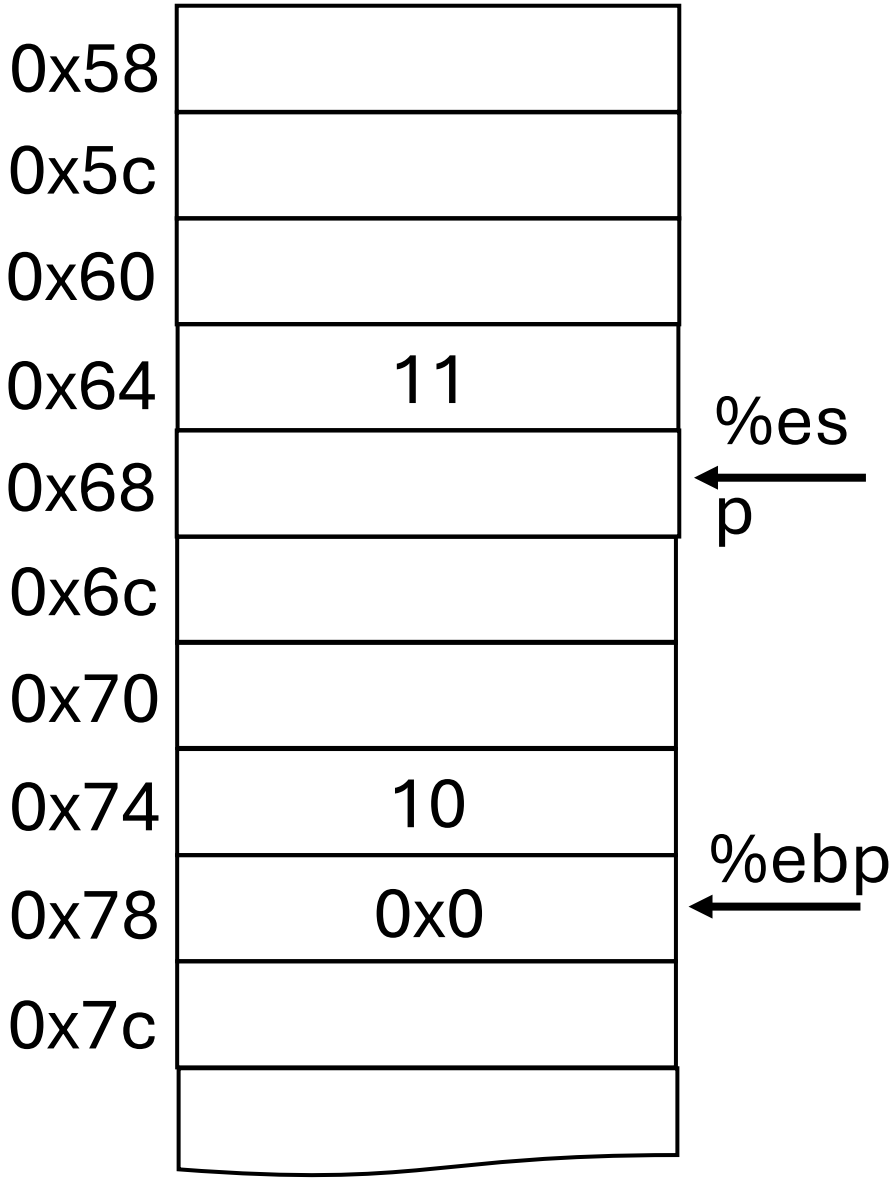


Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

%es	0xffffd068
%ebp	0xffffd078
%eip	0x565561eb

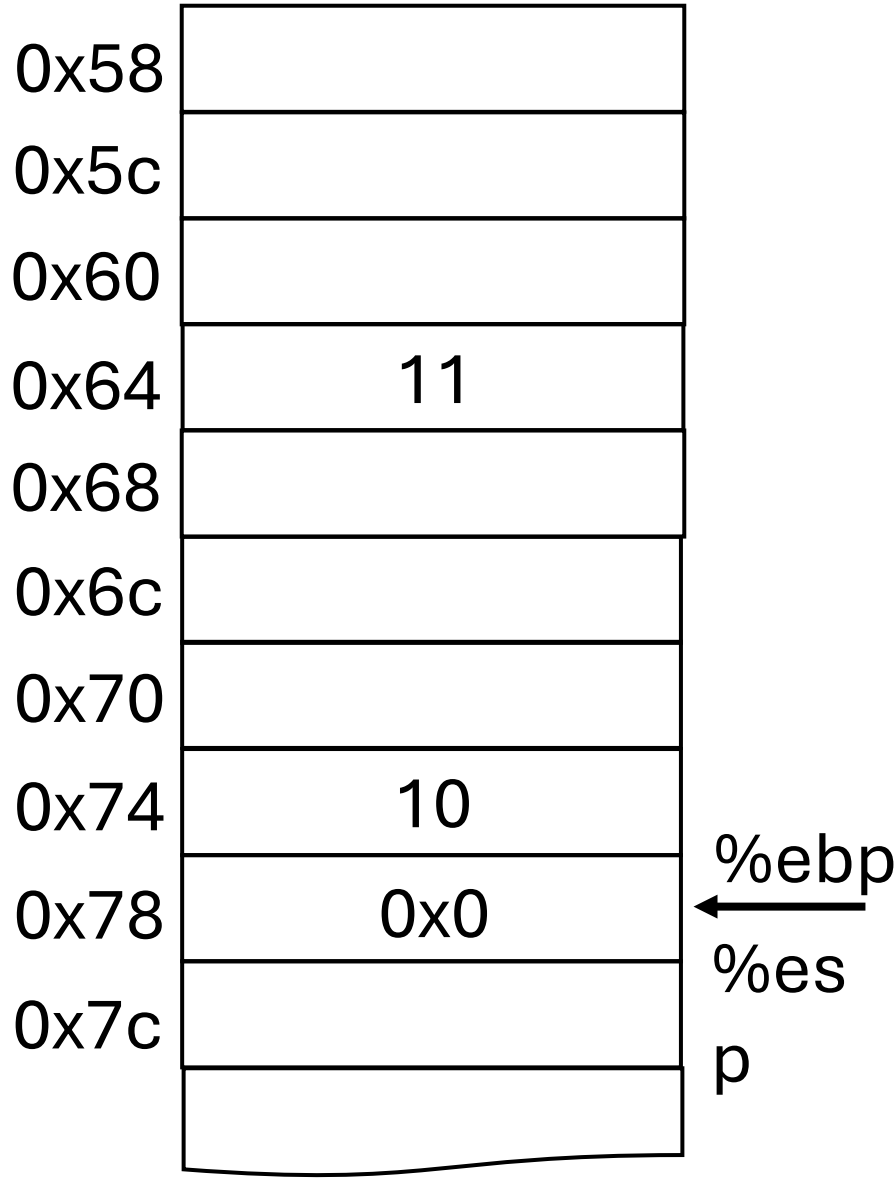


Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        mov %ebp, %esp
        pop %ebp
        ret
```

%esp	0xffffd078
%ebp	0xffffd078
%eip	0x565561eb

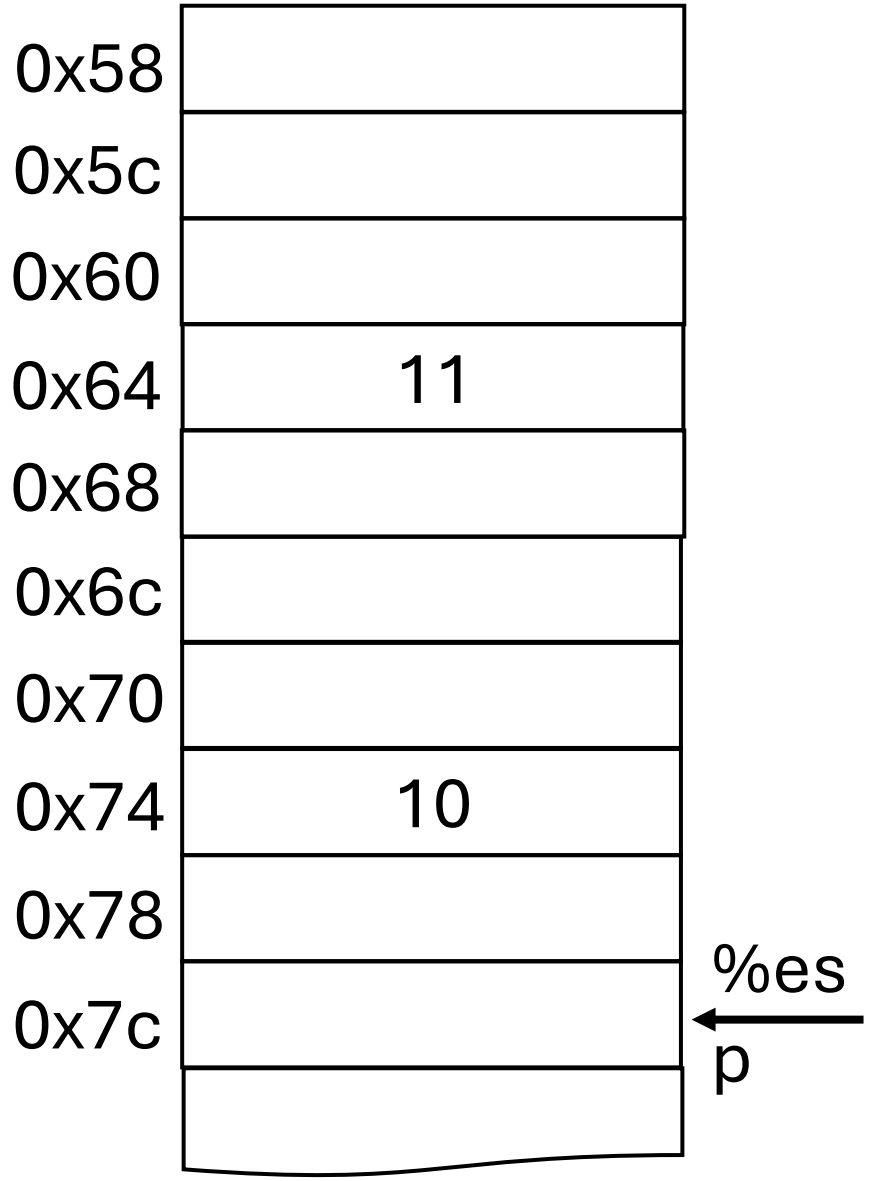


Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

%es	0xffffd07
%eb	0x0
%eip	0x565561eb

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        mov %ebp, %esp
        pop %ebp
        ret
```

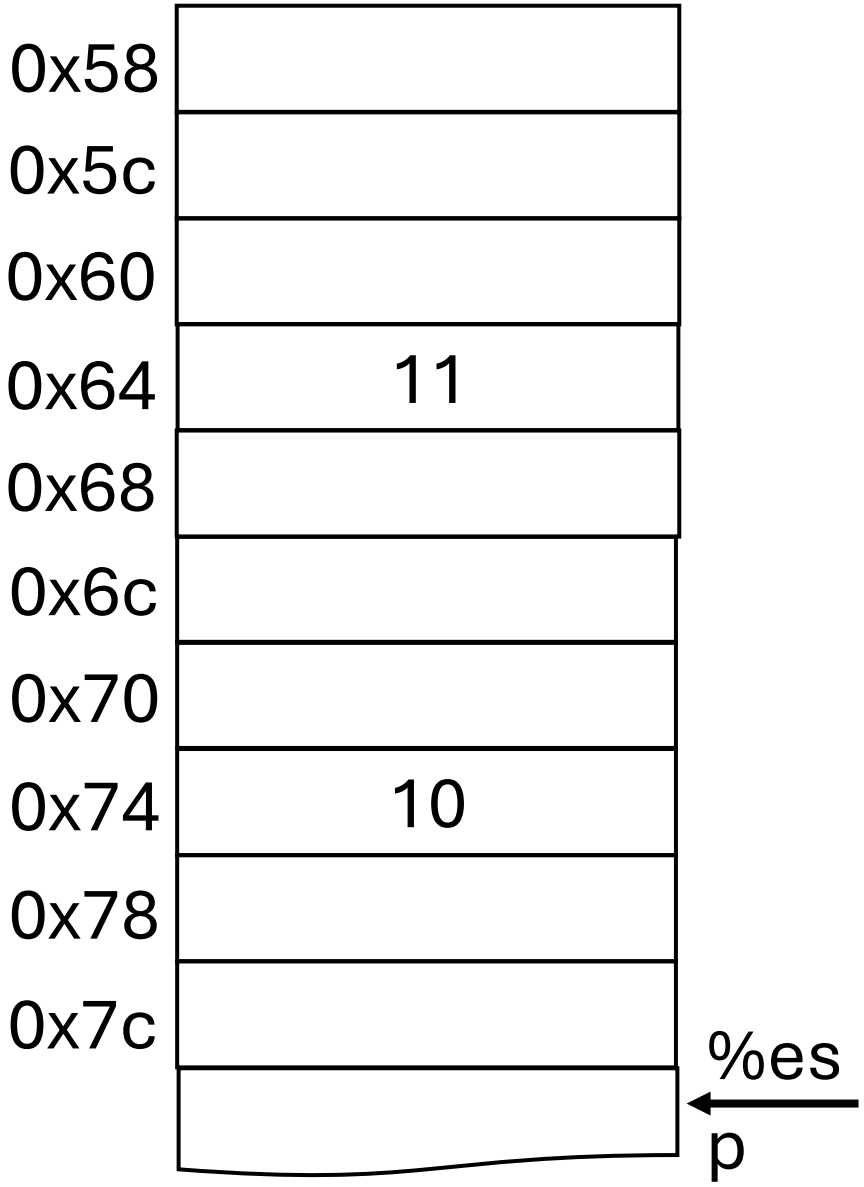


Вызов функции

```c
#include <stdio.h>

void func(int x)
{
    x++;
}

int main()
{
int x = 10;
func(x);
return 0;
}
```

```
func:
        pushl   %ebp
        movl    %esp, %ebp
        addl    $1, 8(%ebp)
        popl    %ebp
        ret
main:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $10, -4(%ebp)
        pushl   -4(%ebp)
        call    func
        addl    $4, %esp
        movl    $0, %eax
        leave
        ret
```

%es	0xffffd08
%eb	0x0
%eip	0xf7de6ee5



Еще один небольшой пример

```
#include <stdio.h>
int main(void)
{
  char x[2];
  scanf("%s", x);
  return 0;
}
```

Алгоритм действий

1. Проверяем порядок следования байт

```
readelf secret -a
```

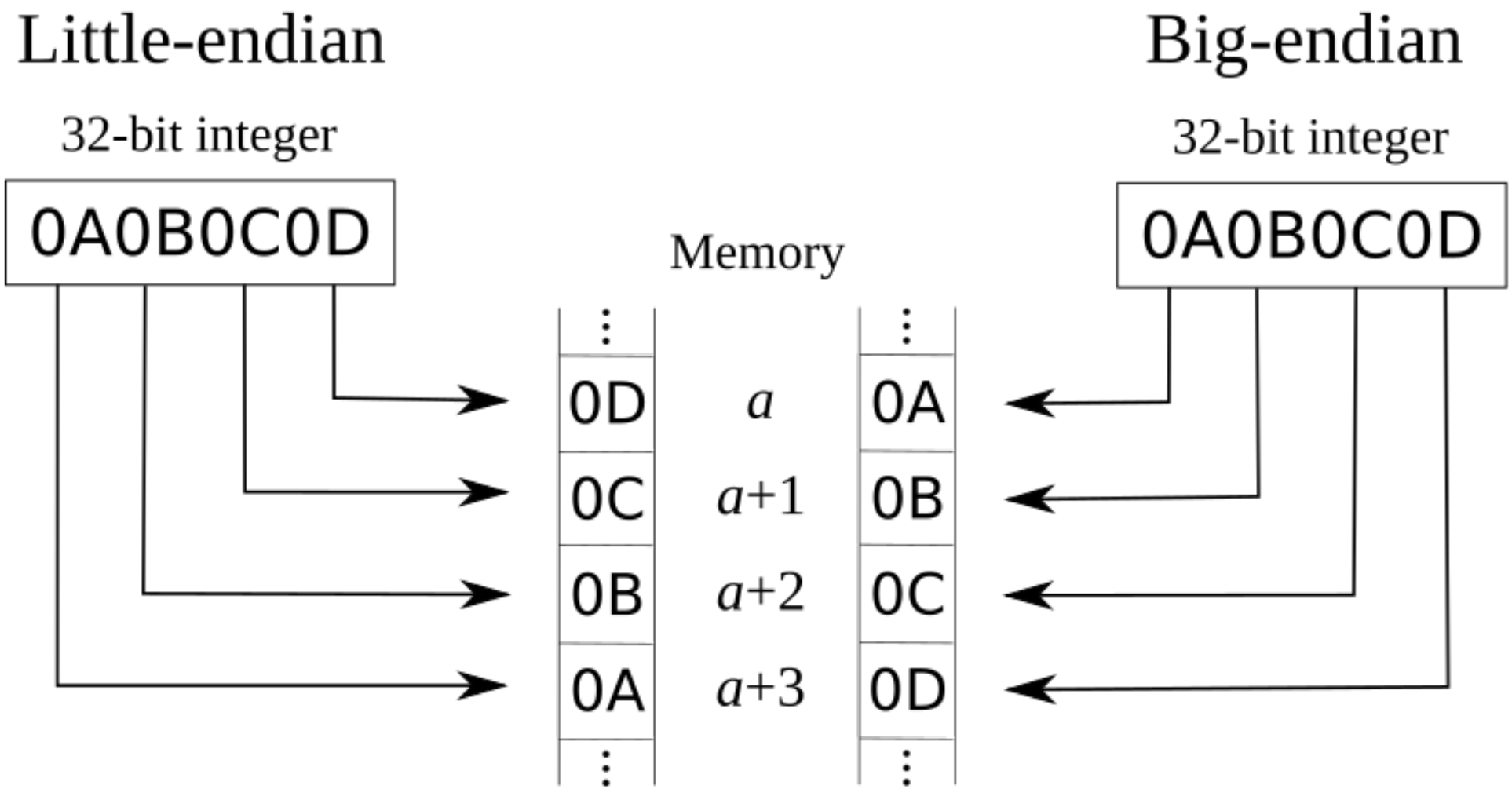
2. Анализируем ассемблерный код

```
objdump secret -d
```

3. Анализируем запущенную программу

```
gdb secret
```

Напоминание



Попытка взлома

```
#include <stdio.h>

char ebuff[] =
"\x31\x32\x33\x34\x35\x36\x37\x38\x39\x30" /*first 10 bytes of junk*/
"\x31\x32\x33\x34\x35\x36\x37\x38\x39\x30" /*next 10 bytes of junk*/
"\x31\x32\x33\x34\x35\x36\x37\x38\x39\x30" /*following 10 bytes of junk*/
"\x31\x32\x33\x34\x35\x36\x37\x38\x39\x30" /*last 10 bytes of junk*/
"\xda\x06\x40\x00\x00\x00\x00\x00" /*address of endGame (little endian)*/
;

int main(void) {
    int i;
    for (i = 0; i < sizeof(ebuff); i++) { /*print each character*/
        printf("%c", ebuff[i]);
    }
    return 0;
}
```

Попытка взлома

```
gcc -o exp exp.c
./exp > exploit
./secret < exploit
 echo $?
```

Мораль!

Instead of:	Use:
`gets(buf)`	`fgets(buf, 12, stdin)`
`scanf("%s", buf)`	`scanf("%12s", buf)`
`strcpy(buf2, buf)`	`strncpy(buf2, buf, 12)`
`strcat(buf2, buf)`	`strncat(buf2, buf, 12)`
`sprintf(buf, "%d", num)`	`snprintf(buf, 12, "%d", num)`

Сегменты и секции

Модульное программирование

Работа с портом

- Настройка порта
- …
- Чтение данных
- …
- Отправка сигнала завершения
- …

Работа с буфером

- Запись данных в буфер
- …
- Чтение данных из буфера
- …

Работа с файлом

- Открытие файла
- …
- Запись данных
- …
- Закрытие файла
- …

main

Анализ данных

…

Удобно работать с программой

Модульное программирование

Работа с портом

- Настройка порта
- …
- Чтение данных
- …
- Отправка сигнала завершения
- …

Работа с буфером

- Запись данных в буфер
- …
- Чтение данных из буфера
- …

Работа с файлом

- Открытие файла
- …
- Запись данных
- …
- Закрытие файла
- …

main

Анализ данных

…

Как бы вы предложили разбить текст программы для удобства работы загрузчика?

Удобно работать с программой

Секции программы

```
section .text
   global _start      ;must be declared for linker (ld)

_start:               ;tells linker entry point
   mov  edx,len       ;message length
   mov  ecx,msg       ;message to write
   mov  ebx,1         ;file descriptor (stdout)
   mov  eax,4         ;system call number (sys_write)
   int  0x80          ;call kernel

   mov  eax,1         ;system call number (sys_exit)
   int  0x80          ;call kernel
```

Программный код

```
section .data
msg db 'Hello, world!', 0xa  ;string to be printed
len equ $ - msg       ;length of the string
```

Данные

Секции программы

```
#include <stdio.h>
#include <stdlib.h>

static int bss[1024];
static int data[1024] = {1024};
static const char rodata[8192] = {"rodata"};

int main(void)
{
	int stack;
	int *heap = (int*)malloc(1024*sizeof(int));
	free(heap);
	return 0;
}
```

Task 099

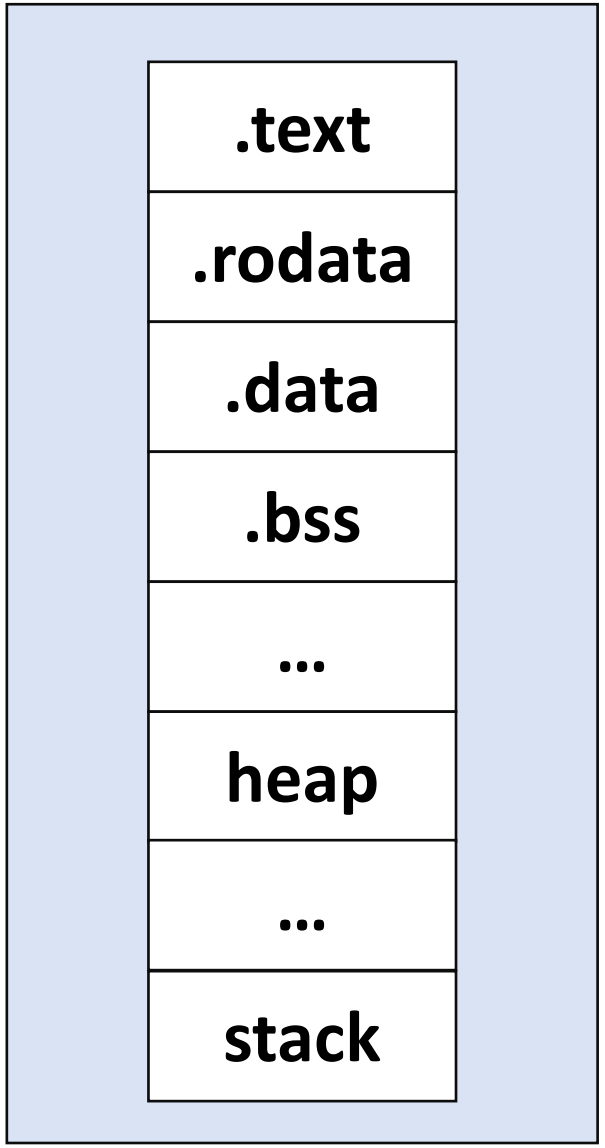
Секции программы

К О Д

```
#include <stdio.h>
#include <stdlib.h>

static int bss[1024];
static int data[1024] = {1024};
static const char rodata[8192] = {"rodata"};

int main(void)
{
    int stack;
    int *heap = (int*)malloc(1024*sizeof(int));
    free(heap);
    return 0;
}
```

- `static int bss[1024];` → Неинициализированная глобальная переменная
- `static int data[1024] = {1024};` → Инициализированная глобальная переменная
- `static const char rodata[8192] = {"rodata"};` → Инициализированная глобальная константа
- `int stack;` → Статическая переменная
- `int *heap = (int*)malloc(1024*sizeof(int));` → Динамический массив

Секции программы



Код программы

Глобальная константы

Инициализированная глобальные переменные

Неинициализированная глобальные переменные

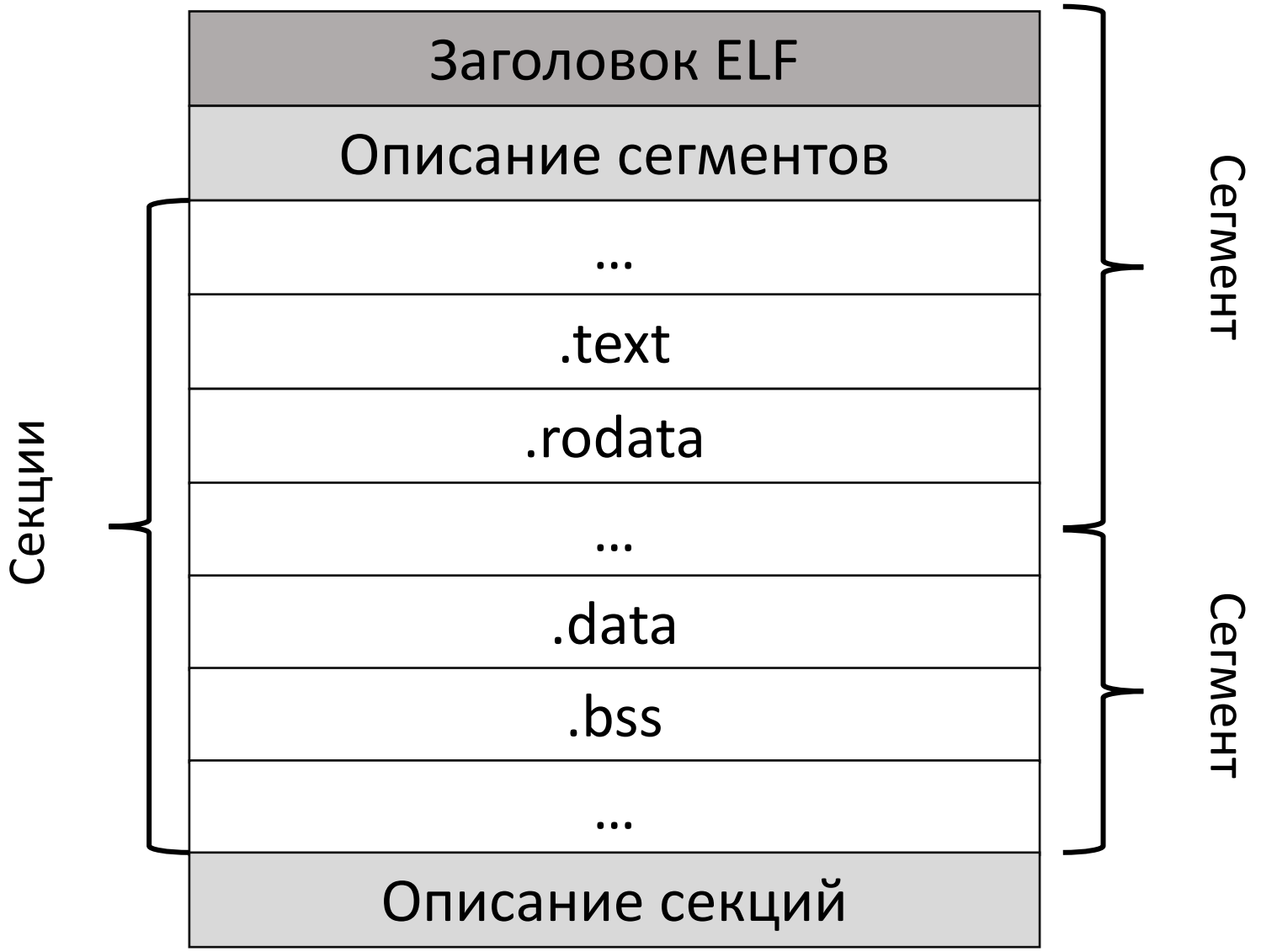
Куча

Стек

Секции и сегменты программы

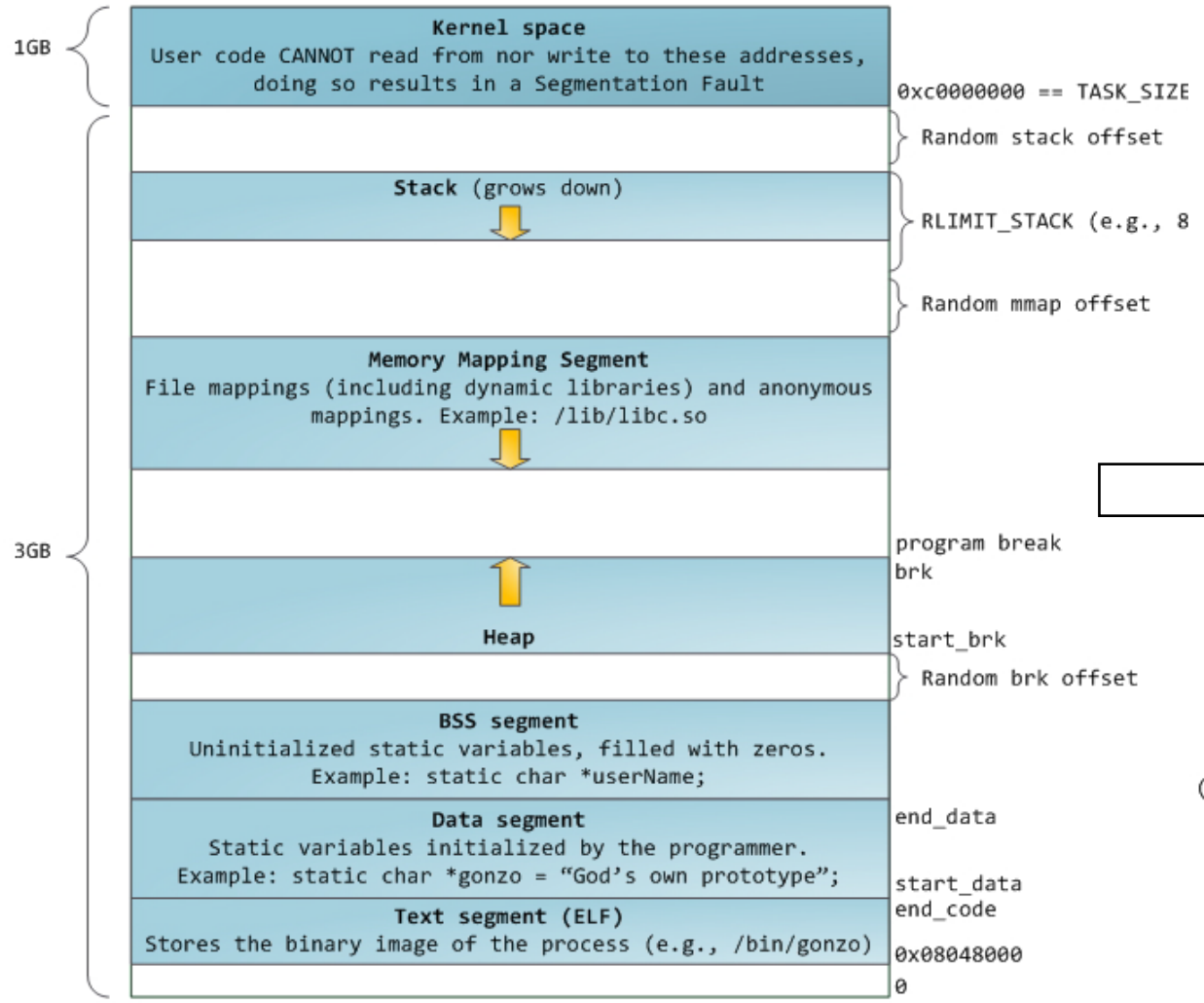
Компоновщик

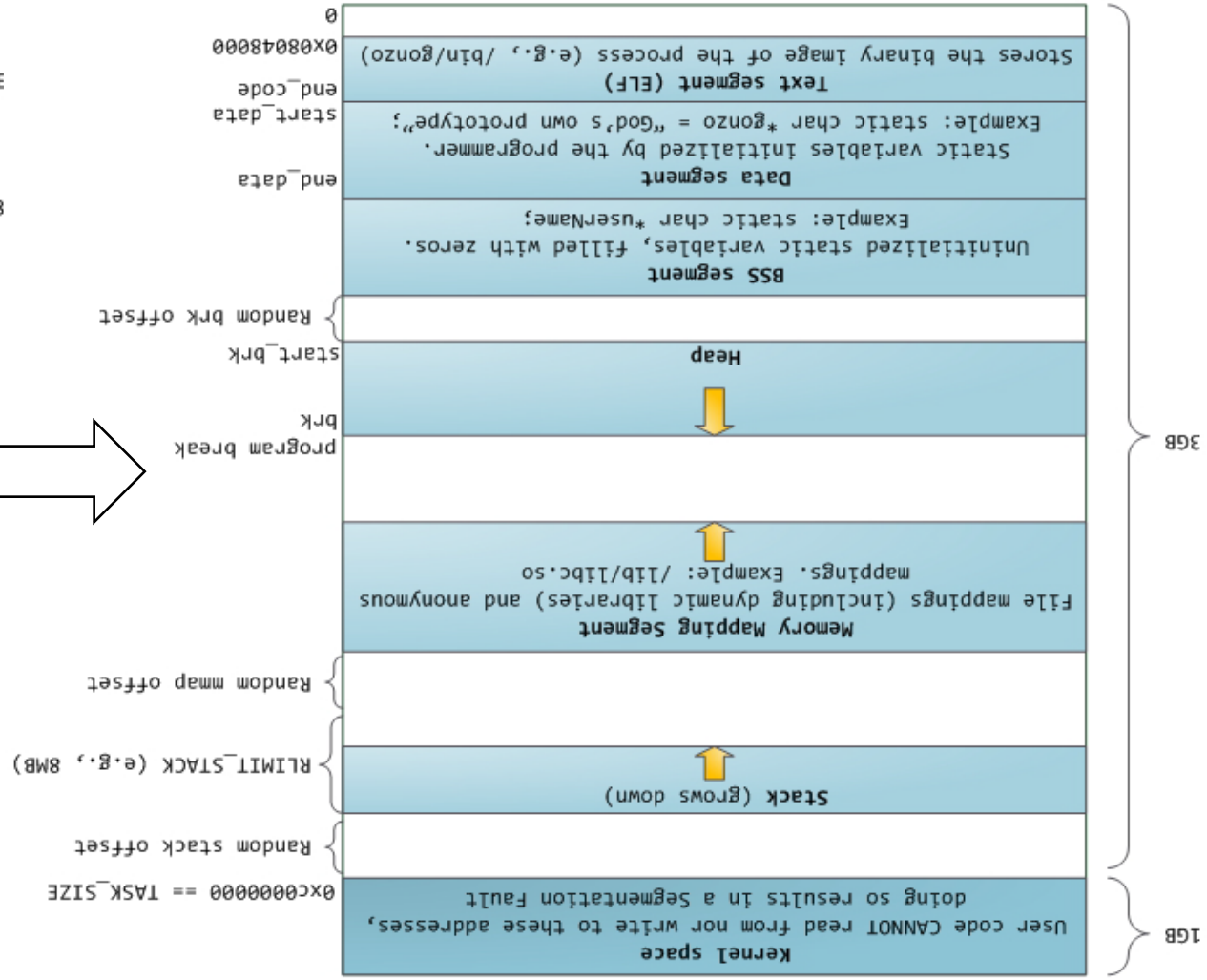
Загрузчик



LOAD

Секции программы





ELF файлы

Что?

Исходный код

↓

Препроцессор

↓

Компилятор

↓

Ассемблер

↓

Компоновщик/ загрузчик

↓

Целевой машинный код

ELF файл

ELF (Executable and Linkable Format) — формат двоичных файлов, определяющий структуру бинарных файлов, библиотек, и файлов ядра

Процесс – программа во время исполнения и все её элементы: адресное пространство, глобальные переменные , регистры, стек, счетчик команд, состояние, открытые файлы, дочерние процессы и т. д

Task 100

Устройство ELF файлов

Основные компоненты:

1) Заголовок
2) Описание секций
3) Описание сегментов

Устройство ELF файлов

Основные компоненты:

1) Заголовок
2) Описание секций
3) Описание сегментов

Заголовок файла

```
typedef struct
{
    unsigned char e_ident[16];
    uint16_t e_type;
    uint16_t e_machine;
    uint32_t e_version;
    uint32_t e_entry;
    uint32_t e_phoff;
    uint32_t e_shoff;
    uint32_t e_flags;
    uint16_t e_ehsize;
    uint16_t e_phentsize;
    uint16_t e_phnum;
    uint16_t e_shentsize;
    uint16_t e_shnum;
    uint16_t e_shstrndx;
} Elf32_Ehdr;
```

Пример

```
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Shared object file)
  Machine:                           Intel 80386
  Version:                           0x1
  Entry point address:               0x1070
  Start of program headers:          52 (bytes into file)
  Start of section headers:          18528 (bytes into file)
  Flags:                             0x0
  Size of this header:               52 (bytes)
  Size of program headers:           32 (bytes)
  Number of program headers:         12
  Size of section headers:           40 (bytes)
  Number of section headers:         31
  Section header string table index: 30
```

readelf memory -h

Устройство ELF файлов

Основные компоненты:

1) Заголовок
2) Описание секций
3) Описание сегментов

Информация, хранящаяся в ELF-файле, организована в секции. Каждая секция имеет свое уникальное имя. Некоторые секции хранят служебную информацию ELF-файла (например, таблицы строк), другие секции хранят отладочную информацию, третьи секции хранят код или данные программы.

Таблица заголовков секций

```
typedef struct
{
        uint32_t sh_name;
        uint32_t sh_type;
        uint32_t sh_flags;
        uint32_t sh_addr;
        uint32_t sh_offset;
        uint32_t sh_size;
        uint32_t sh_link;
        uint32_t sh_info;
        uint32_t sh_addralign;
        uint32_t sh_entsize;
 } Elf32_Shdr;
```

Пример

```
Section Headers:
  [Nr] Name          Type        Addr        Off     Size    ES   Flg   Lk   Inf   Al
  [ 0]               NULL        00000000    000000  000000  00        0    0     0
  [ 1] .interp       PROGBITS    000001b4    0001b4  000013  00   A    0    0     1
  ...
  [12] .init         PROGBITS    00001000    001000  000024  00   AX   0    0     4
...
  [16] .text         PROGBITS    00001070    001070  0001e9  00   AX   0    0     16
....
  [18] .rodata       PROGBITS    00002000    002000  000e1c  00   A    0    0     32
....
  [25] .data         PROGBITS    00005000    004000  000010  00   WA   0    0     4
  [26] .bss          NOBITS      00005010    004010  000008  00   WA   0    0     4
  ....
```

readelf memory -S

Устройство ELF файлов

Основные компоненты:

1) Заголовок
2) Описание секций
3) Описание сегментов

Таблица заголовков программы содержит информацию, необходимую для загрузки программы на выполнение.

Таблица заголовков программы

```
typedef struct
{
        uint32_t p_type;
        Elf32_Off p_offset;
        Elf32_Addr p_vaddr;
        Elf32_Addr p_paddr;
        uint32_t p_filesz;
        uint32_t p_memsz;
        uint32_t p_flags;
        uint32_t p_align;
} Elf32_Phdr;
```

Пример

Program Headers:

	Type	Offset	VirtAddr	PhysAddr	FileSiz	MemSiz	Flg	Align
0	PHDR	0x000034	0x00000034	0x00000034	0x00180	0x00180	R	0x4
1	INTERP	0x0001b4	0x000001b4	0x000001b4	0x00013	0x00013	R	0x1
2	LOAD	0x000000	0x00000000	0x00000000	0x003d4	0x003d4	R	0x1000
3	LOAD	0x001000	0x00001000	0x00001000	0x00274	0x00274	R E	0x1000
4	LOAD	0x002000	0x00002000	0x00002000	0x00f94	0x00f94	R	0x1000
5	LOAD	0x003edc	0x00004edc	0x00004edc	0x00134	0x0013c	RW	0x1000
6	DYNAMIC	0x003ee4	0x00004ee4	0x00004ee4	0x000f8	0x000f8	RW	0x4

…

readelf memory -l

Пример

Program Headers:

	Type	Offset	VirtAddr	PhysAddr	FileSiz	MemSiz	Flg	Align
0	PHDR	0x000034	0x00000034	0x00000034	0x00180	0x00180	R	0x4
1	INTERP	0x0001b4	0x000001b4	0x000001b4	0x00013	0x00013	R	0x1
2	LOAD	0x000000	0x00000000	0x00000000	0x003d4	0x003d4	R	0x1000
3	LOAD	0x001000	0x00001000	0x00001000	0x00274	0x00274	R E	0x1000
4	LOAD	0x002000	0x00002000	0x00002000	0x00f94	0x00f94	R	0x1000
5	LOAD	0x003edc	0x00004edc	0x00004edc	0x00134	0x0013c	RW	0x1000
6	DYNAMIC	0x003ee4	0x00004ee4	0x00004ee4	0x000f8	0x000f8	RW	0x4

…

Segment Sections...

00

01 .interp

02 .interp .note.gnu.build-id .note.gnu.property .note.ABI-tag .gnu.hash .dynsym .dynstr .gnu.version

03 .init .plt .plt.got .plt.sec **.text** .fini

04 **.rodata** .eh_frame_hdr .eh_frame

05 .init_array .fini_array .dynamic .got **.data .bss**

06 .dynamic

…

Пример со своим ELF

```
git clone https://github.com/SergeyBalabaev/Elective-C-Programming-Language
```

Elective -> lesson6 -> ELF

```
sudo apt install xxd -y
chmod +x make.sh
./make.sh
./Prog
```

Пример со своим ELF

```
./make.sh
./Prog
```

Измените программу так, чтобы она выводила ваши имя, фамилию и отчество (целиком)

ASCII Code Chart

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Напоминание

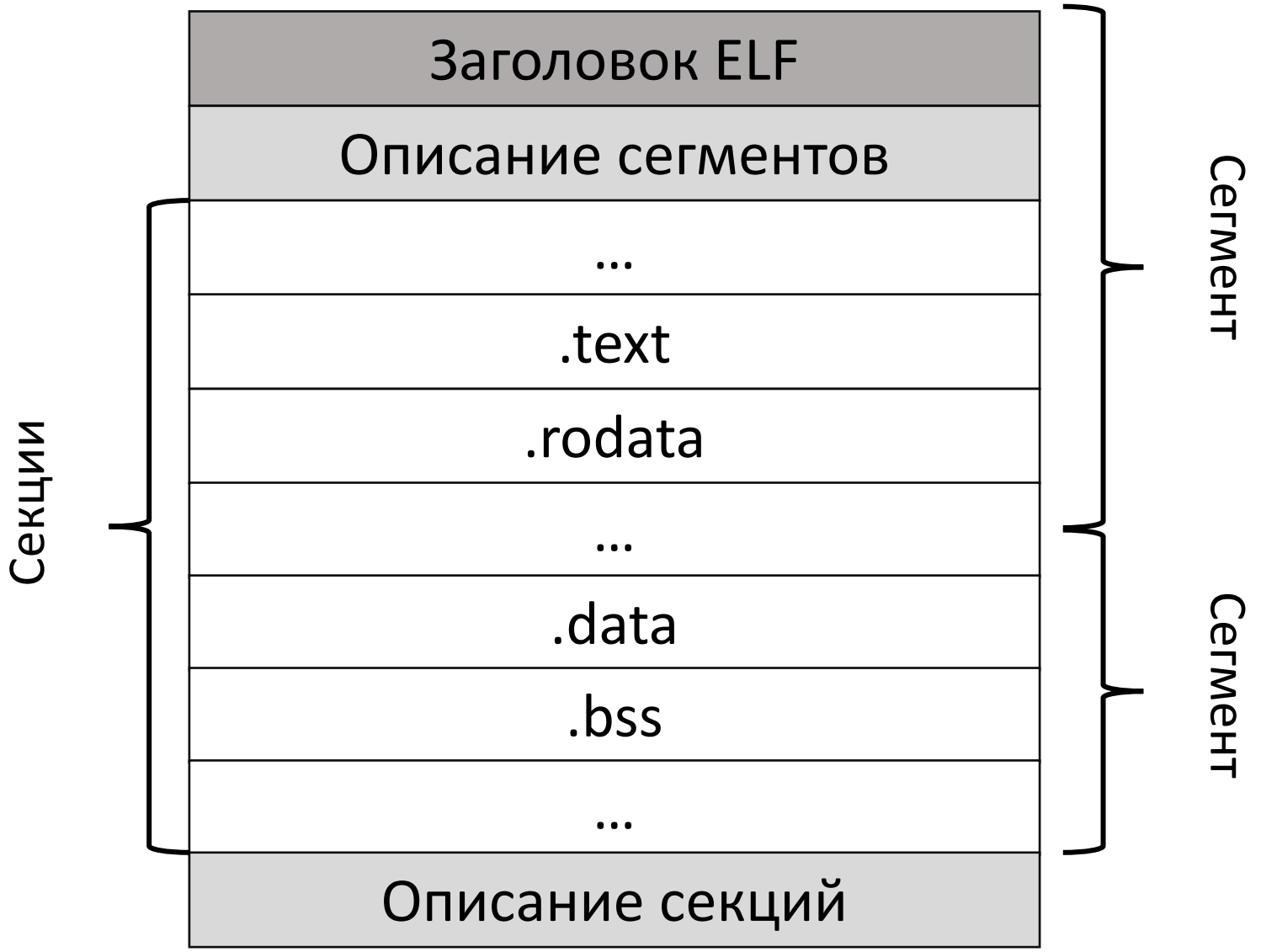
1) Память поделена на **страницы**
2) Преобразование:

Логический адрес -> Линейный адрес -> Физический адрес

Напоминание

Компоновщик

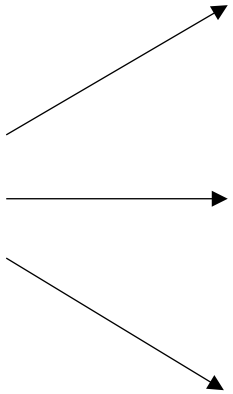
Загрузчик



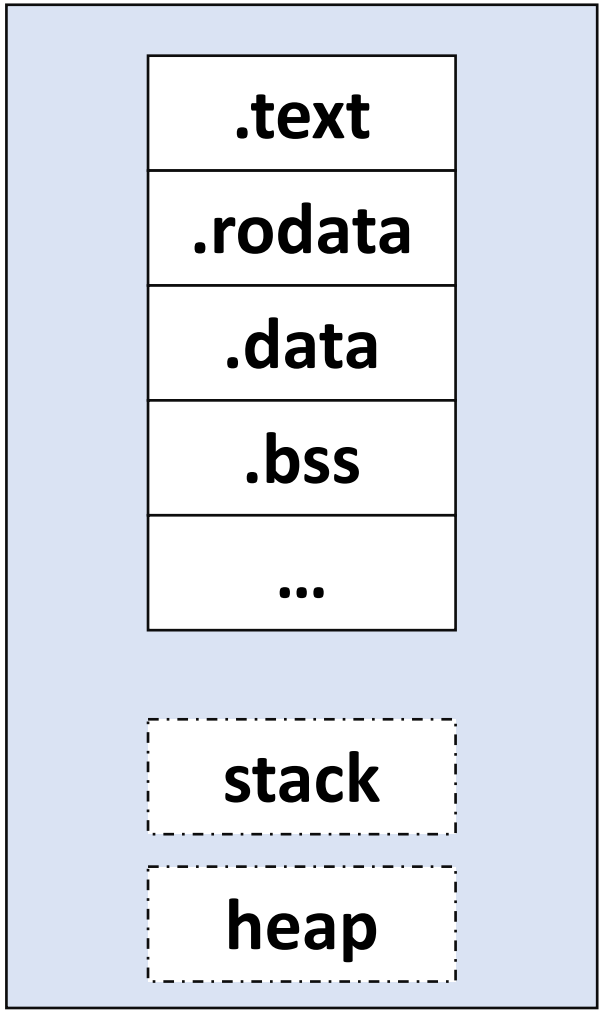
LOAD

Напоминание

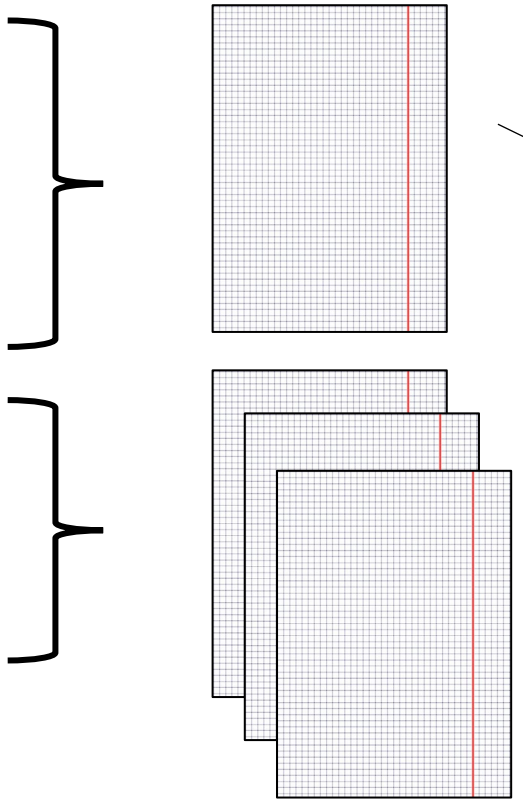
```
int main(void)
{
    ///
    return 0;
}
```

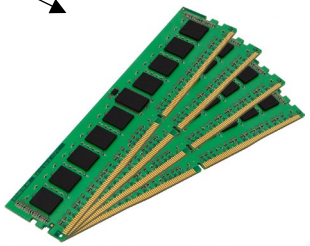


Секции



Сегменты





Секции программы